

推理、规划与反思范式

智能体需要在推理、行动、观察与反思之间建立可控制的求解结构。

- Reasoning 形成判断
- Acting 改变环境
- Observation 更新状态
- Reflection 改进下一轮

一个看似简单的任务

比较高校生成式 AI 政策时，同一模型和工具可以采用完全不同的执行结构。

- 直接回答
- 边搜索边判断
- 先规划再执行
- 失败后修改计划

回答正确不等于任务完成

Agent 任务同时评价最终结果、信息来源、行动过程和约束满足情况。

- 结果是否满足目标
- 来源是否可验证
- 过程是否越权
- 预算和停止条件是否满足

五个核心概念

推理、规划、行动、观察和反思共同构成 Agent 的动态决策系统。

- 推理: 形成判断
- 规划: 组织步骤
- 行动: 调用外部能力
- 观察: 读取反馈
- 反思: 评价过程

决策范式的两条轴

是否预先规划、是否依据环境反馈调整，是选择执行范式的两条关键轴。

- ReAct: 强反馈
- Plan-and-Execute: 强规划
- Replanning: 规划加反馈
- Direct: 最小结构

Direct Response

当输入完整且输出可以一次生成时，Direct 是成本最低的正确结构。

- 单次生成
- 无外部行动
- 无显式循环
- 适合摘要、分类和提取

Reactive 范式

Reactive 系统根据当前状态立即行动，响应快但可能缺少长期一致性。

- 关注当前状态
- 局部反馈快
- 长期目标容易丢失
- 需要边界和停止条件

Direct 与 Reactive 的局限

信息不完整或任务较长时，Direct 和局部 Reactive 都可能积累错误。

- 无法主动补充事实
- 缺少全局进度
- 重复调用工具
- 预算可能失控

为什么只有推理或只有行动都不够

封闭推理缺少外部事实，盲目行动缺少目标关联，二者必须形成闭环。

- 推理维护方向
- 行动获取新信息
- 观察修正假设
- 循环必须可停止

ReAct 的基本结构

每轮依据最新 Observation 决定下一步 Action，并持续检查完成条件。

```
while not done:  
    action = reason(state, observation)  
    observation = execute(action)  
    state = update(state, observation)
```

ReAct 执行示例

搜索结果不可信时，Agent 根据 Observation 调整来源范围并继续完成任务。

- 识别信息缺口
- 选择搜索工具
- 验证来源
- 更新下一步行动

ReAct 的优势

ReAct 让工具结果真实改变后续行为，并逐步缩小未知范围。

- 适合开放式任务
- 轨迹可以观察
- 及时处理反馈
- 关键未知逐轮收敛

ReAct 的典型失败

没有进度和预算治理时，ReAct 可能循环、重复工具调用或偏离目标。

- 重复搜索
- 过早停止
- 观察未进入状态
- 缺少最大步数

不要把 ReAct 等同于完整思维链

工程审计需要决策证据、行动摘要和状态变化，而不是无限暴露内部生成文本。

- 记录当前子目标
- 记录行动理由摘要
- 保留工具参数
- 保存 Observation 和状态变化

为什么需要先规划

具有依赖、中间产物和完成标准的复杂任务需要显式计划。

- 识别子目标
- 声明依赖
- 定义产物
- 设置验收条件

Plan-and-Execute 结构

Planner 生成可执行计划，Executor 执行当前步骤并更新状态。

```
plan = planner(goal)
for step in plan:
    result = executor(step, state)
    state = update(state, result)
```


一个合格的计划应包含什么

每一步都应明确输入、输出、依赖与完成条件。

- 输入可获得
- 输出可保存
- 依赖顺序有效
- 完成可以检查

Plan-and-Execute 的优势和风险

计划保持全局结构，但错误假设会被执行器机械地放大。

- 优势: 进度可追踪
- 优势: 支持人工审核
- 风险: 初始计划失效
- 风险: 规划成本过高

计划为什么会失效

工具失败、环境变化、预算压力和新约束都会让剩余计划失效。

- 工具超时
- 证据推翻假设
- 用户追加目标
- 权限或安全约束变化

Replanning 不是重新从头生成

正确的 Replanning 保留已完成产物，只修改受新信息影响的剩余步骤。

- 冻结已完成步骤
- 标记失效假设
- 替换受影响步骤
- 记录修改理由

Replanning 触发器

触发器必须显式记录并映射到确定的恢复策略。

- 执行失败
- Observation 冲突
- 目标变化
- 预算越界
- 安全策略阻断

什么是反思

反思把结果和环境反馈转化为下一次行动的可执行改进。

- 评价当前结果
- 识别失败原因
- 形成改进动作
- 决定继续、重试或停止

四种容易混淆的结构

Self-Critique、Self-Refine、Reflexion 和 Evaluator-Optimizer 的反馈来源与记忆范围不同。

- Self-Critique: 评价
- Self-Refine: 评价后修改
- Reflexion: 保留经验
- Evaluator-Optimizer: 角色分离

Evaluator-Optimizer

Evaluator 根据 Rubric 输出通过或修订建议，Optimizer 只处理可验证缺陷。

```
evaluation = evaluator(output, rubric)
if not evaluation.passed:
    output = optimizer(output, evaluation.feedback)
```


为什么评价器可能失败

评价器可能共享生成器盲点，或因标准模糊给出无法执行的反馈。

- Rubric 不明确
- 共享知识盲点
- 缺少外部验证
- 偏好掩盖硬约束

反思什么时候值得使用

高价值、可评价且允许迭代的任务更适合反思。

- 存在明确质量标准
- 错误代价较高
- 反馈能转成改进
- 有最大迭代预算

单一路径推理的局限

早期判断错误会污染后续步骤，需要在重要决策点保留多个候选。

- 比较多个方案
- 允许回溯
- 外部验证候选
- 避免局部最优

Tree of Thoughts

ToT 生成、评价和剪枝多个中间状态，并在必要时回溯。

```
frontier = expand(state)
scored = evaluate(frontier)
frontier = keep_top_k(scored)
```

搜索式推理的成本

候选数量随深度快速增长，因此必须限制宽度、深度与模型预算。

- Beam Search
- Top-K 保留
- 规则剪枝
- 低成本模型初筛

六种范式比较

范式选择应从任务特征、反馈强度和主要失败模式出发。

- Direct
- Reactive
- ReAct
- Plan-and-Execute
- Replanning

选择范式的决策表

先采用最简单可行结构，再只增加能够解决真实失败模式的机制。

- 输入完整则 Direct
- 需要工具则 ReAct
- 步骤稳定则 Planning
- 计划易失效则 Replanning
- 可评价则 Evaluator

一个现实系统通常是混合的

混合架构不是堆叠复杂性，而是让每一层解决一个明确问题。

Goal

- > Plan-and-Execute
- > ReAct tool loop
- > Replanning on failure
- > Evaluator quality gate
- > Human review for high risk